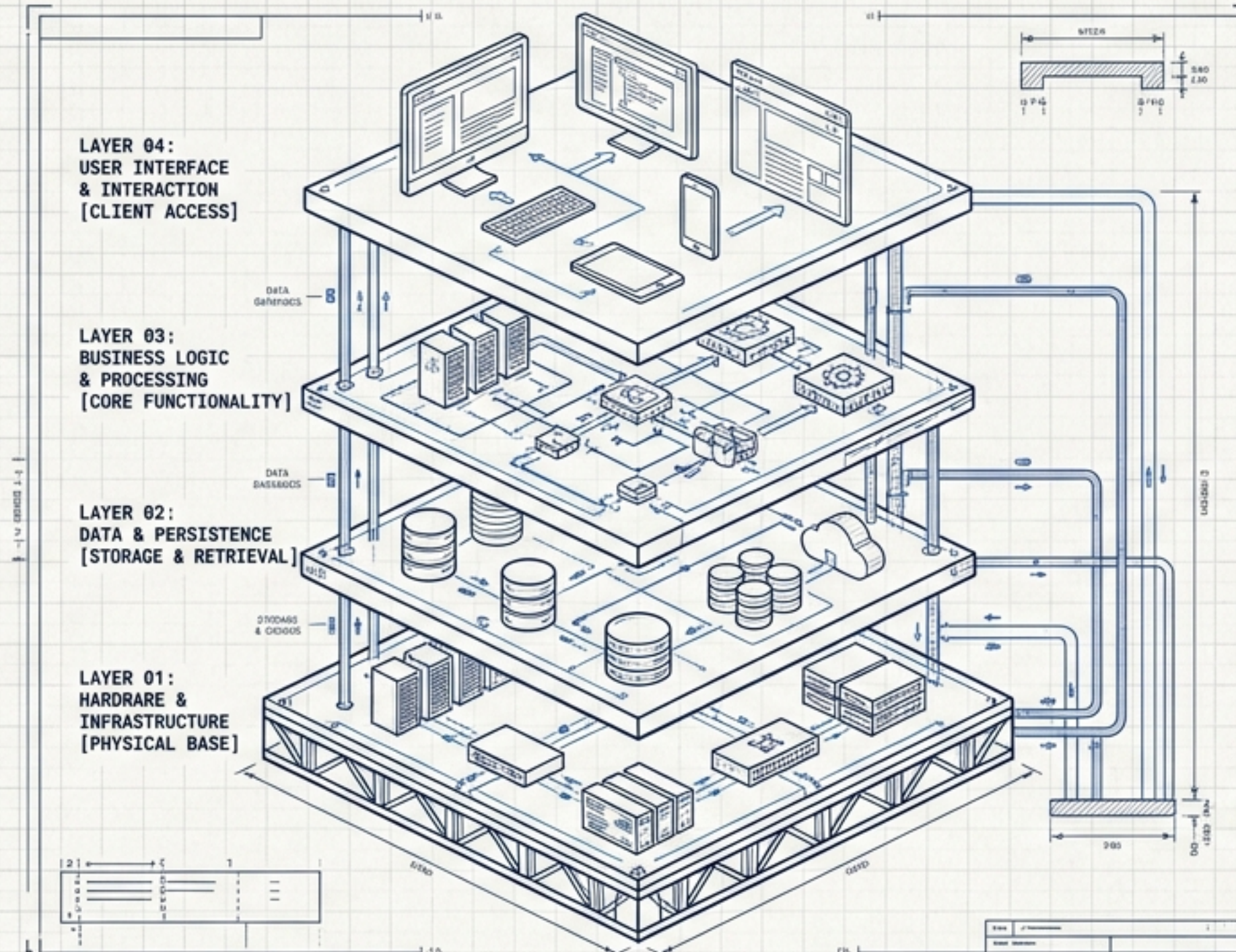


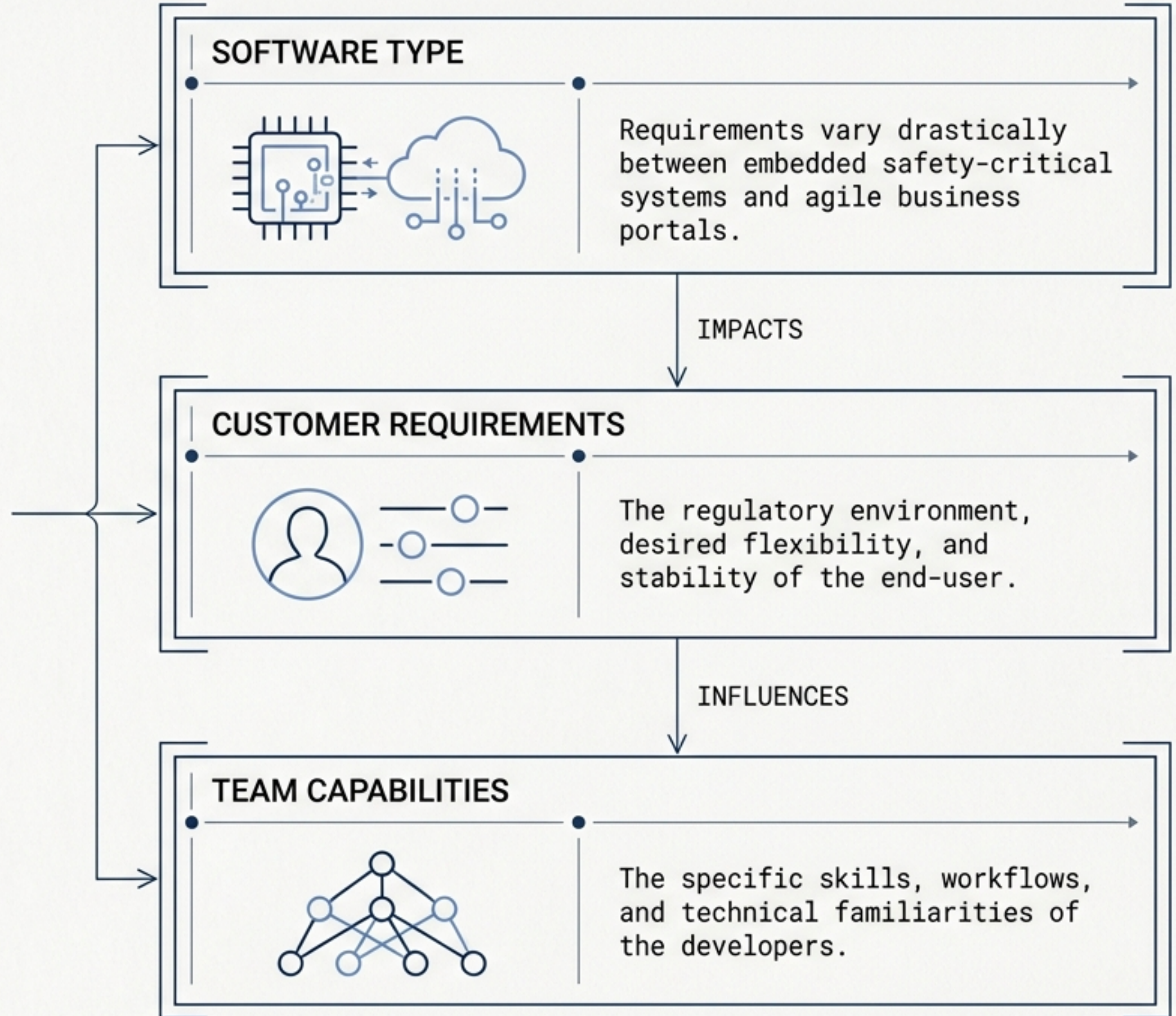
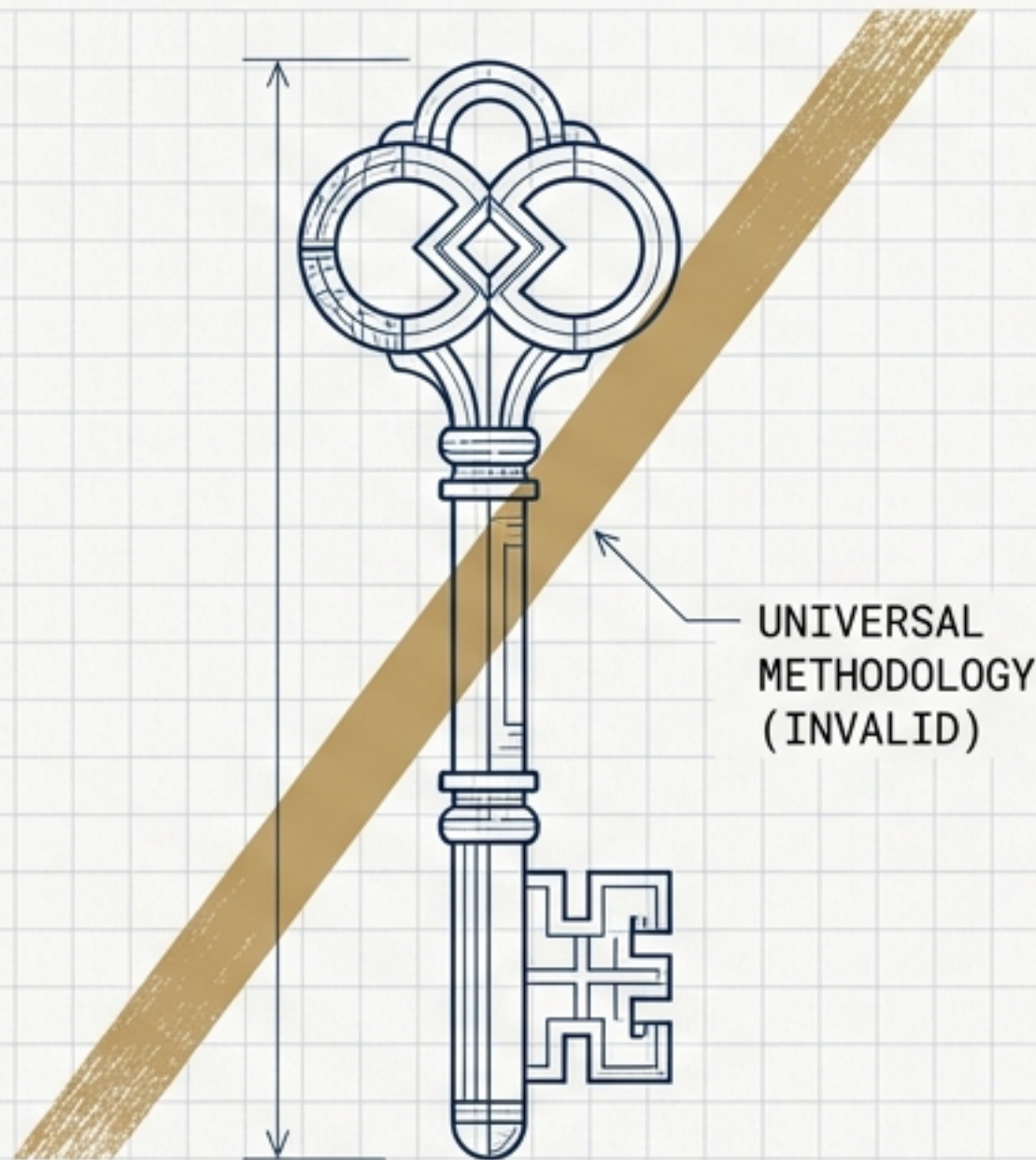


ARCHITECTING SOFTWARE PROCESS

A Diagnostic Guide to Software Development Life Cycle (SDLC) Paradigms

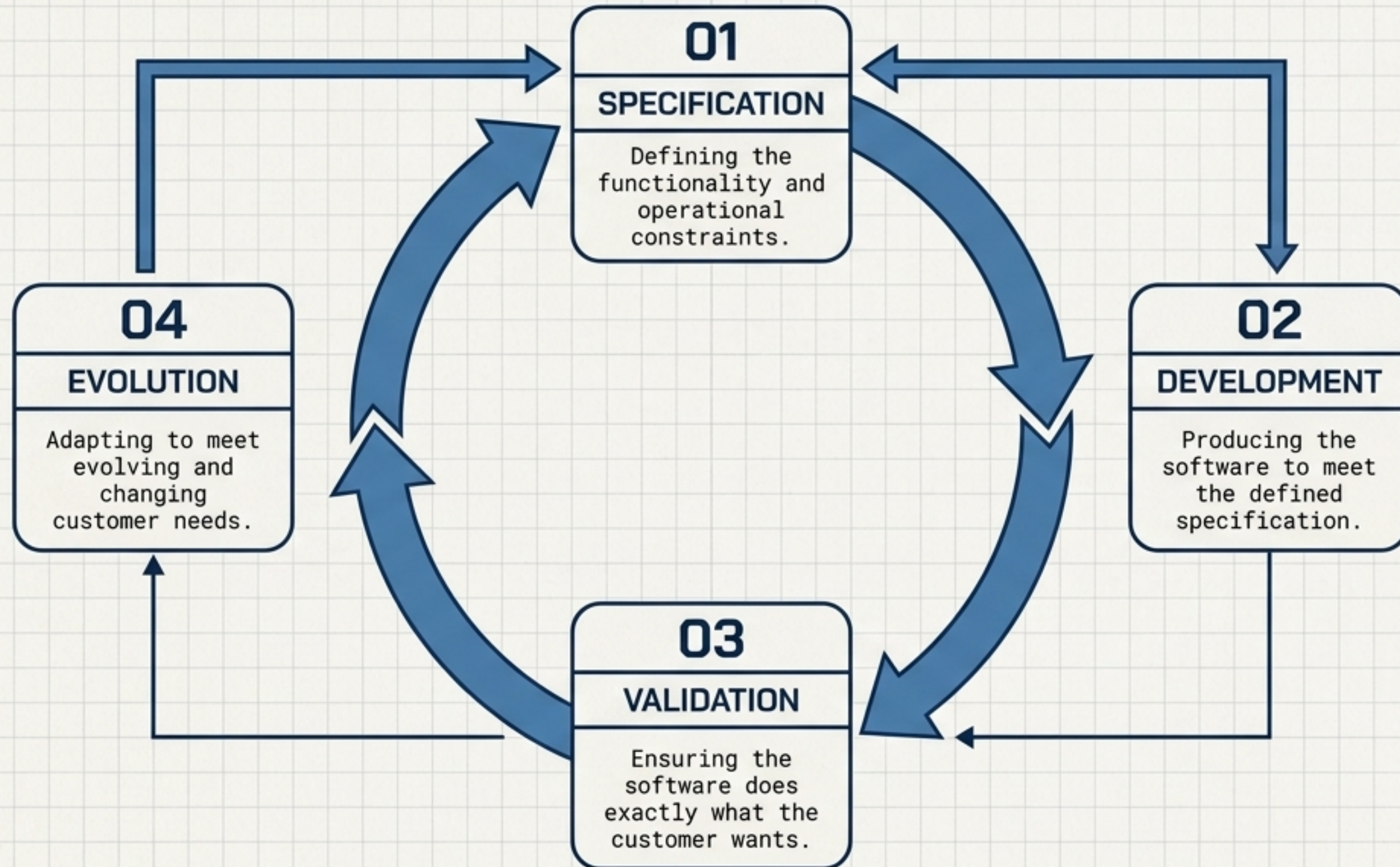
THE MYTH OF THE UNIVERSAL PROCESS

Software is highly variable. Consequently, there is no universally applicable software engineering method.



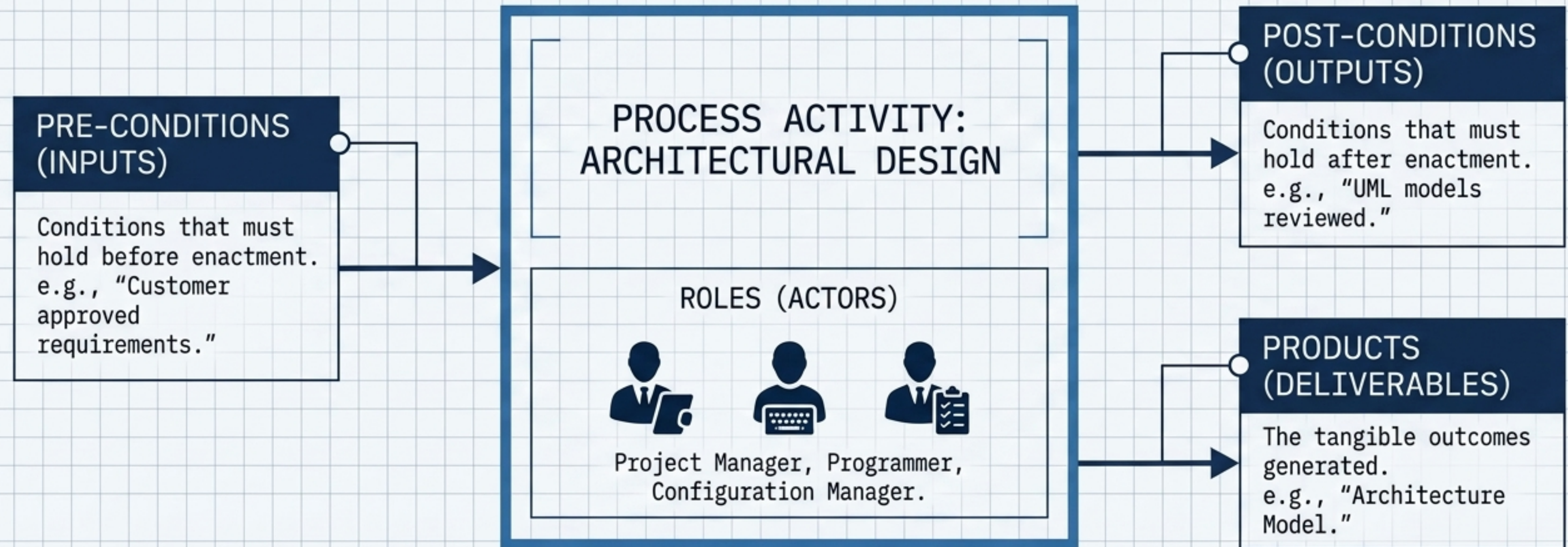
THE FOUR FUNDAMENTAL CONSTANTS

Every software process must contain these four genetic building blocks



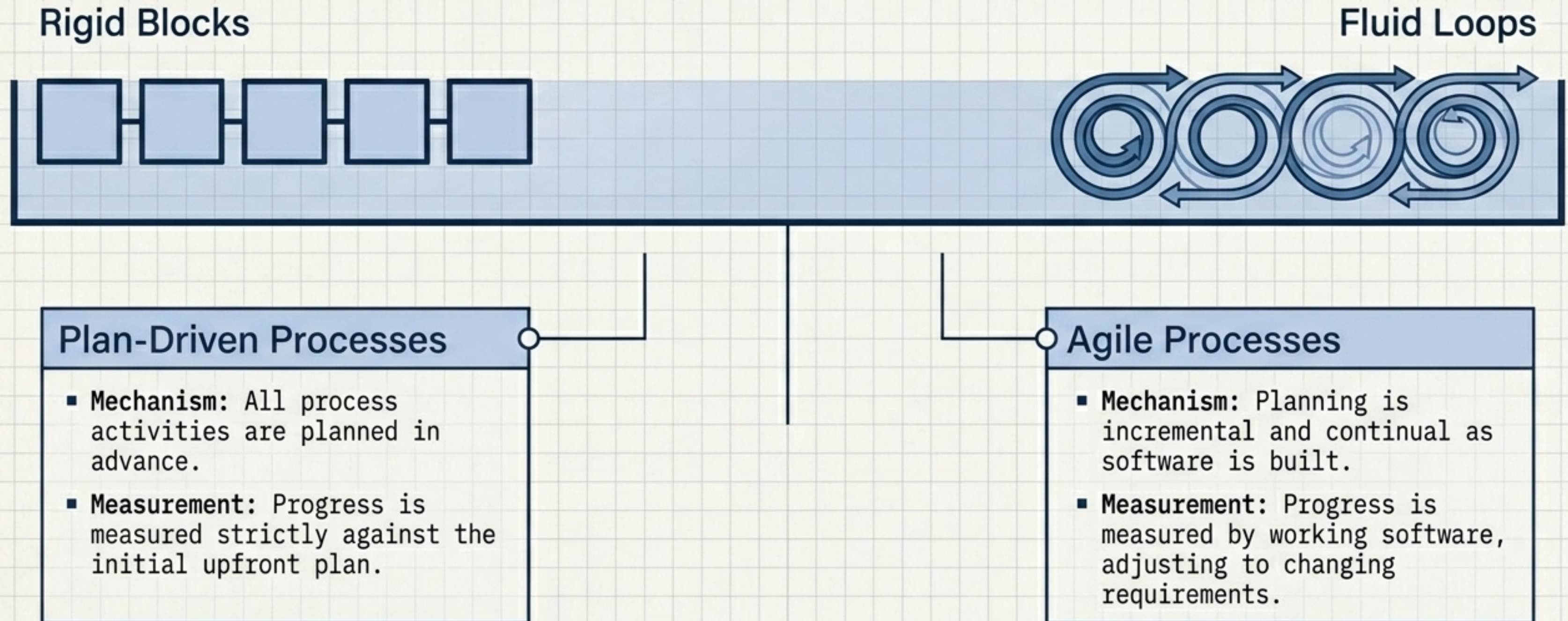
ANATOMY OF A PROCESS ACTIVITY

A schematic teardown of a single generic node.



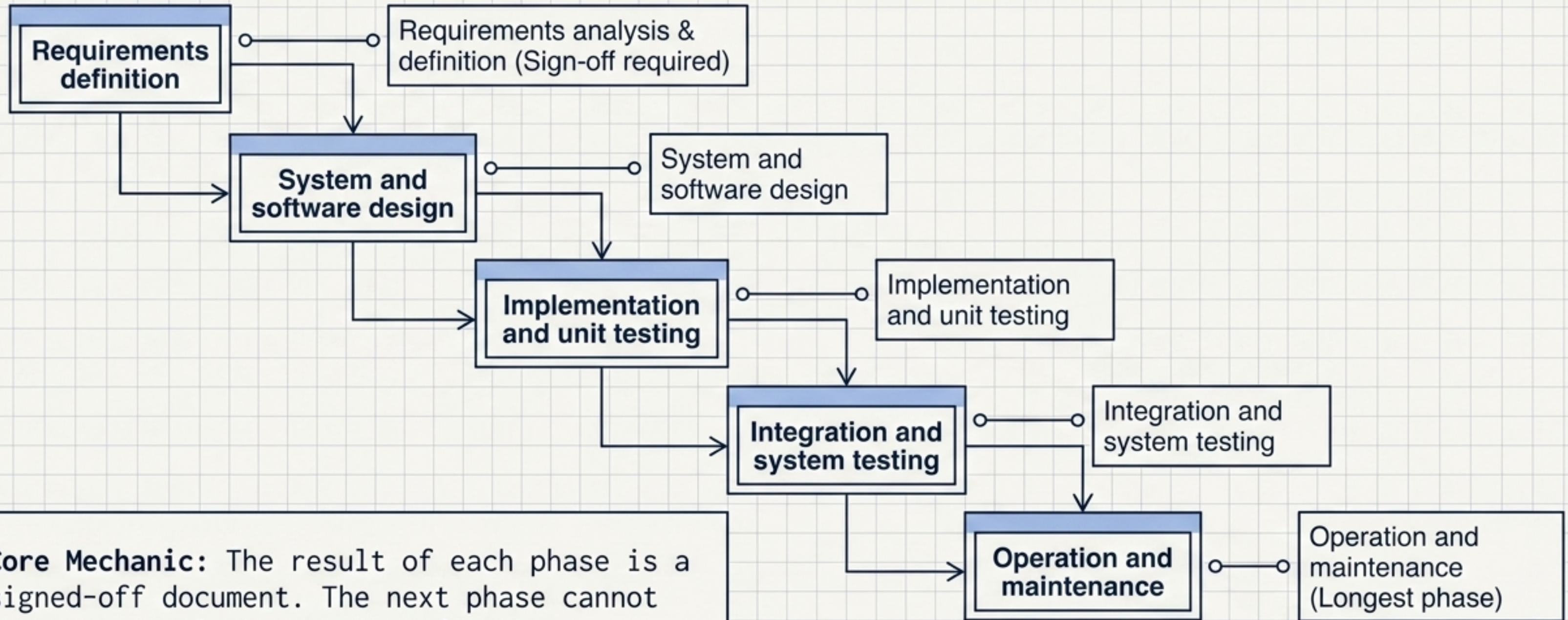
THE PLANNING SPECTRUM

The primary axis on which all process models are evaluated.



Paradigm 1: The Waterfall Model

[Plan-Driven]

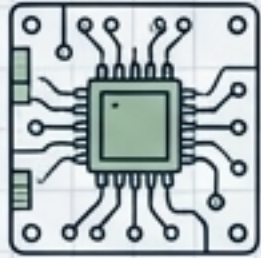


Core Mechanic: The result of each phase is a signed-off document. The next phase cannot commence until the previous finishes.

Waterfall Diagnostics

When to enforce rigidity, and where it breaks down.

Ideal Use Cases



Embedded Systems

Software interfacing with inflexible hardware cannot delay functional decisions.



Critical Systems

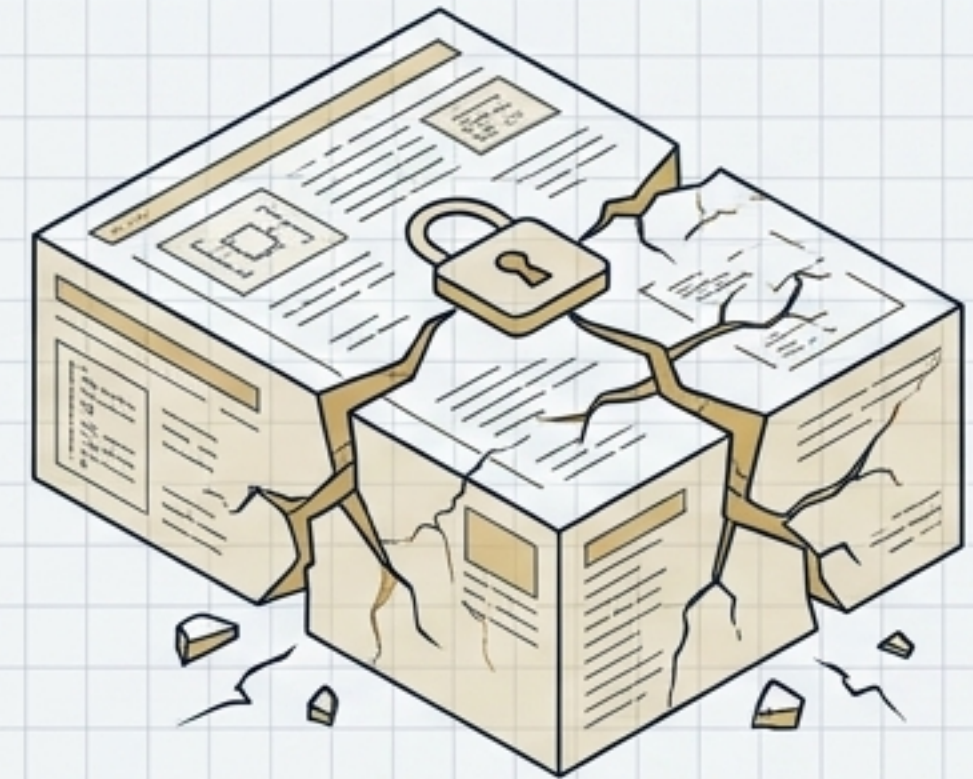
Safety/security analysis requires complete, upfront specification blueprints.



Large Multi-Partner Systems

Independent companies developing subsystems require rigid, stable interfaces.

The Primary Vulnerability

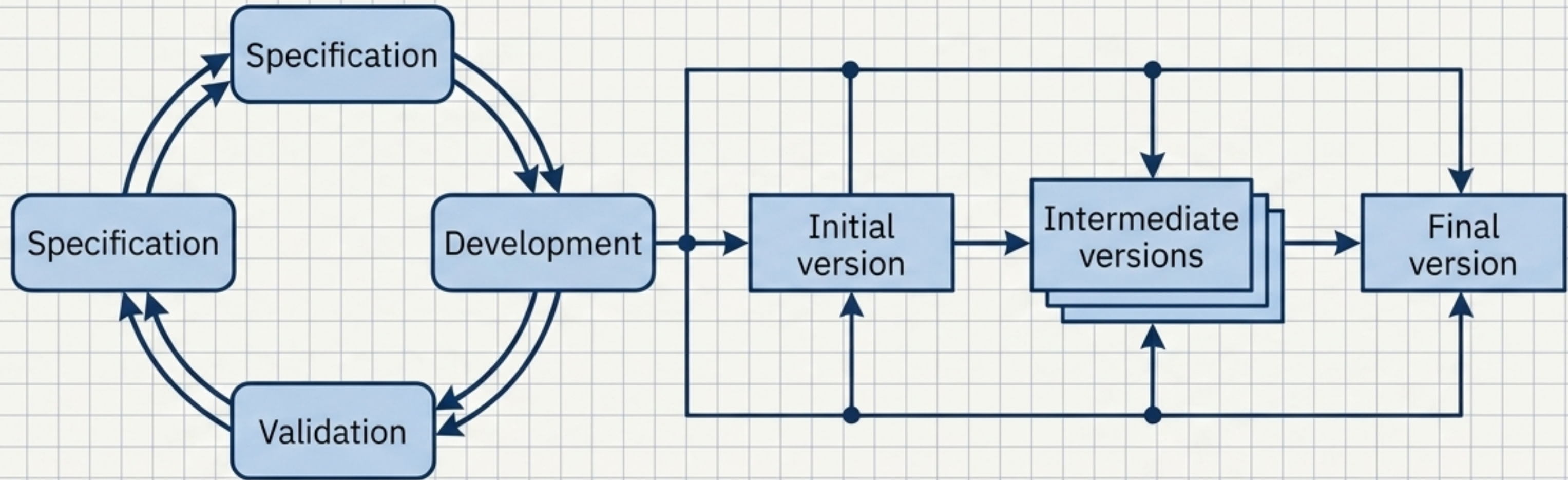


Premature Freezing: Specifications are locked early to avoid delays. This results in badly structured systems via “implementation tricks” to circumvent design flaws discovered too late in the cascade.

Paradigm 2: Incremental Development

[Agile-Leaning]

[Plan-Driven Hybrid]



Core Mechanic: Specification, development, and validation are interleaved with rapid feedback. The system is iteratively developed as a series of versions, evaluating core functionality early.

Incremental Diagnostics

Evaluating the trade-offs of iterative loops.

Advantages

- ✓ **Cost of Change:** Drastically reduced. Requires significantly less analysis and documentation rework compared to Waterfall.
- ✓ **Customer Feedback:** Stakeholders evaluate real, working demonstrations early rather than judging progress from dry design documents.
- ✓ **Early Delivery:** Customers extract value from the software before 100% of the planned functionality is complete.

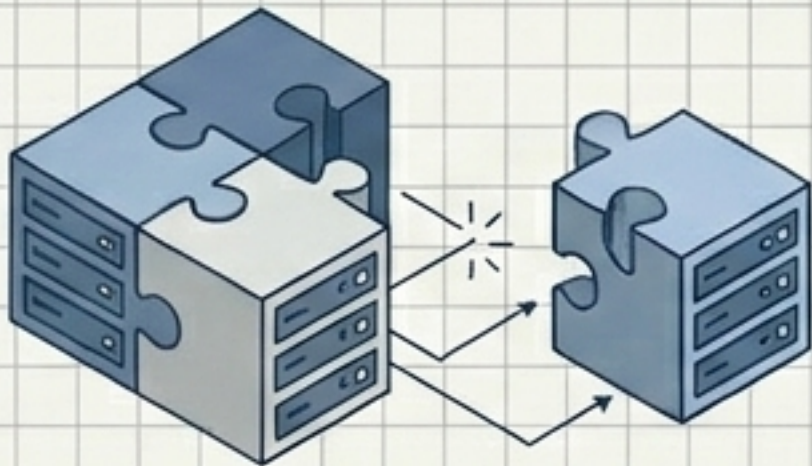
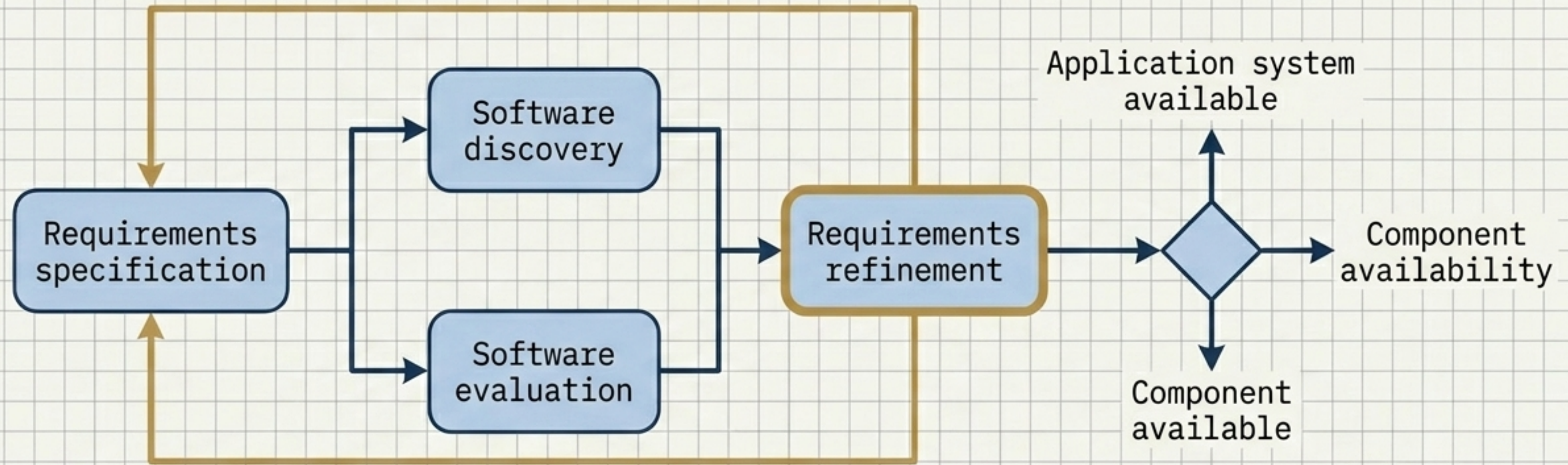
Management & Technical Frictions

- ⚠ **Process Invisibility:** Managers lack regular, formal signed-off deliverables to measure conventional progress.
- ⚠ **Structural Degradation:** Continual adjustments lead to messy code. Requires relentless "refactoring" to prevent the underlying architecture from collapsing under new additions.



Paradigm 3: Integration and Configuration

[Reuse-Oriented]



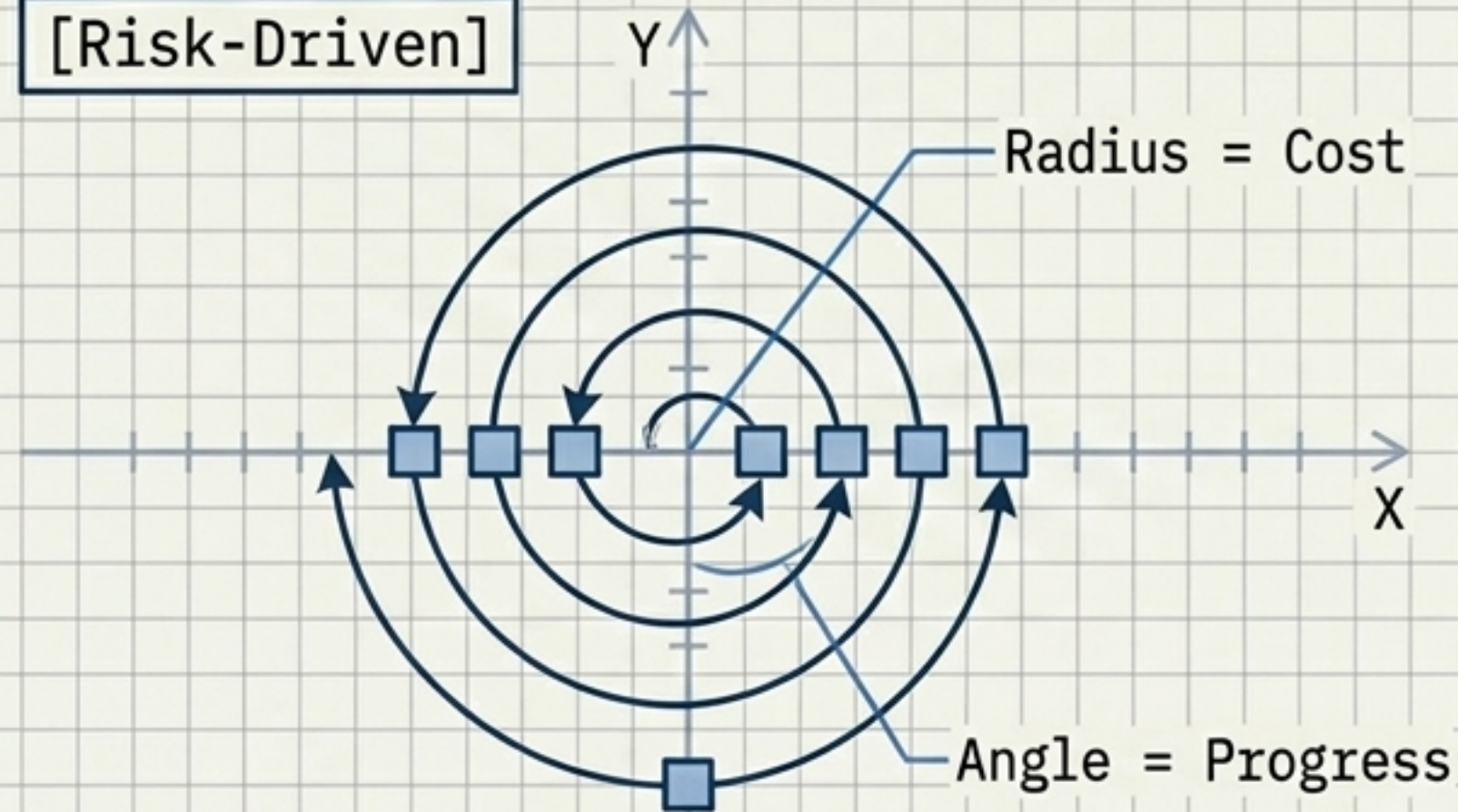
The Crucial Pivot: Requirements Refinement
Unlike bespoke models, requirements are explicitly modified to fit the off-the-shelf components available. If a component cannot be adapted, the goals themselves must be altered.

Advanced Meta-Frameworks

Frameworks that attempt to solve the limitations of the core paradigms.

Boehm's Spiral Model

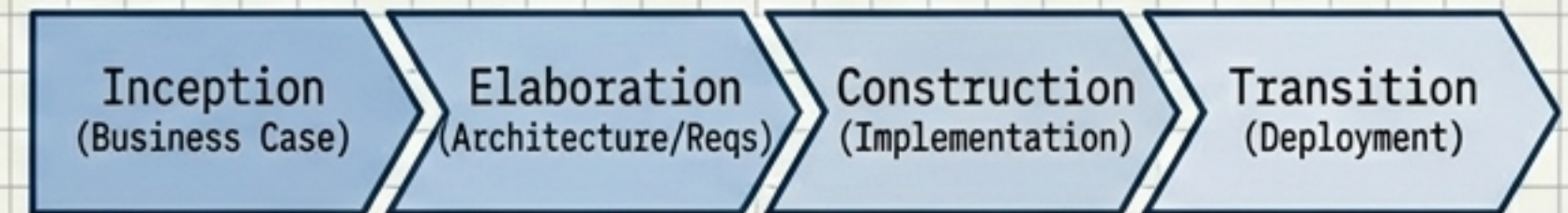
[Risk-Driven]



Advantage: Combines change avoidance with change tolerance by forcing explicit Risk Assessment activities on every rotation.

Rational Unified Process (RUP)

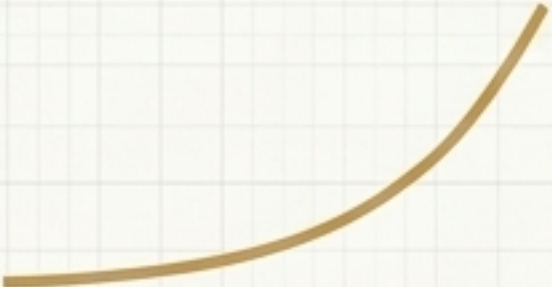
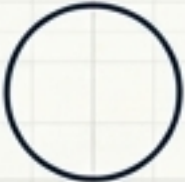
[Universal Attempt]



Brings together elements of all models into a single flexible framework, supporting both prototyping and incremental delivery.

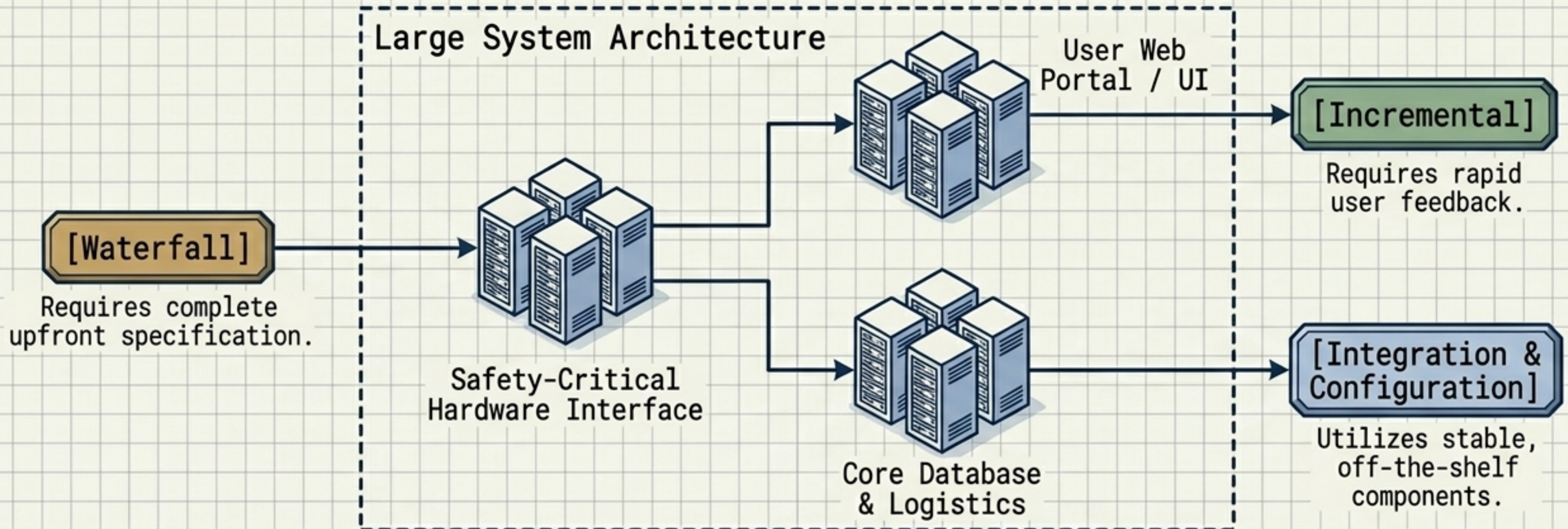
The SDLC Diagnostic Matrix

Comparative synthesis of the three core paradigms.

	Cost of Change	Early Visibility	Ideal Target	Primary Vulnerability
Waterfall		 Poor	Embedded / Safety-Critical	Premature Freezing
Incremental		 Excellent	Business Systems / Products	Structural Degradation
Integration & Config		 Good	Rapid Delivery Business Apps	Requirements Compromise

The Hybrid Reality: Large Systems Engineering

In practice, large-scale systems combine multiple models based on individual subsystem constraints.



There is no universal software process—only context-specific architecture.